Aufgabe 1: Drehfreudig?

Team-ID: 00563

Team-Name: Matthieu Dang

Bearbeiter dieser Aufgabe:
Matthieu Dang

Inhaltsverzeichnis

1 Lösungsidee **1**
- 1.1 Durchsuchung des Baumes . . . 1
- 1.2 Repräsentation der Knoten und Blätter . . . 2

2 Umsetzung **2**
- 2.1 Klasse Knoten . . . 2
- 2.2 Zerlegung der Teilbaum-String in ihre Teilbäume . . . 2
- 2.3 Rekursive Durchsuchung des Baumes . . . 3
- 2.4 Ausführung des Programms . . . 3

3 Werkzeuge **3**

4 Beispiele **4**
- 4.1 Konsolenausgaben . . . 4
- 4.2 Bilder . . . 5

5 Quellcode **5**
- 5.1 Klasse für Knoten . . . 5
- 5.2 Zerlegung der Teilbaum-String in ihre Teilbäume . . . 6
- 5.3 Rekursive Durchsuchung des Baumes . . . 6
- 5.4 Prüfung der Drehfreudigkeit . . . 6

1 Lösungsidee

In dieser Aufgabe soll zurückgegeben werden, ob ein Baum drehfreudig ist. Das bedeutet, dass die Rechtecke um die Blätter vom umgedrehten Baum dieselbe Höhe und Breite, wie die des ursprünglichen Baumes besitzen. Ein drehfreudiger Baum besitzt also eine symmetrische Blätteranordnung, da wenn man den Baum umkehrt, die Reihenfolge der Blätter verkehrt wird.

1.1 Durchsuchung des Baumes

Man benötigt also eine Liste von allen Blättern in der richtigen Reihenfolge, also von Links nach Rechts, oder andersrum sortiert, was hier aber keinen Unterschied macht (Symmetrie). Die Drehfreudigkeit eines Baumes prüft man folglich durch einen Vergleich der Gleichheit mit der verkehrten Liste.

Um diese Liste der Blätter zu ermitteln, durchsucht man den Baum vom Wurzelknoten aus, bis man auf alle Blätter stößt, also es keinen unerforschten Knoten mehr gibt. Diese Suche habe ich rekursiv gestaltet, sodass man von einem Knoten aus alle Kindknoten und Blätter ermittelt. Diese Kindknoten werden dann

selbst alle auf ihre Kindknoten und Blätter durchsucht usw. Gefundene Blätter werden dann direkt zur Liste aller Blätter hinzugefügt.

Wenn man die rekursive Suche immer an dem am weitesten links stehenden Kindknoten beginnt und sich dann nach rechts arbeitet, dann muss die Liste aller Blätter auch von links nach rechts geordnet sein, da der linke Kindknoten vollständig erforscht wird bevor man zu den rechten übergeht.

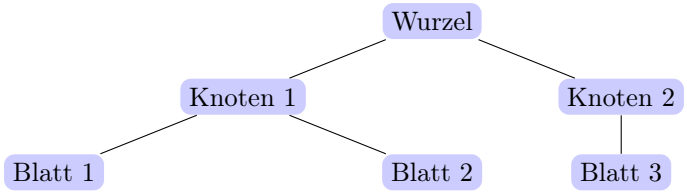


Abb. 1: Der Baum besitzt drei Blätter. Knoten 1 wird als erstes durchsucht, der Blatt 1 und 2 beinhaltet, anschließend kommt Knoten 2 mit Blatt 3. Die Liste der Blätter ist also: [Blatt 1, Blatt 2, Blatt 3]

1.2 Repräsentation der Knoten und Blätter

Blätter müssen bei dem Prüfen der Drehfreudigkeit dieselbe Tiefe und Breite, wie ihr Gegenstück besitzen. Es wird also eine Klasse für Knoten (und Blätter) erstellt, die als Attribute Tiefe und Breite besitzen. Außerdem kommt dazu der Teilbaum, der den Teilbaum des jeweiligen Knotens speichert. Ein Blatt besitzt keinen Teilbaum, dieser wäre dann leer.

Die Tiefe eines Kindknotens ist immer eine Ebene tiefer, als die des Knotens selber. Die Höhe des Rechtecks um Blätter ist die Differenz zwischen der maximalen Tiefe des Baumes und die Tiefe des Blattes. Wenn die Tiefe von zwei Blättern gleich ist, ist die Höhe von dem Rechteck deshalb auch gleich, denn die maximale Tiefe des Baumes ist überall gleich.

Alle Kindknoten eines Knotens besitzen zusammen dieselbe Breite, wie der Knoten selbst. Die Breite eines Kindknotens ist also $\frac{rel.Knotenbreite}{Kindanzahl}$. Wenn ein Knoten n Kinder hat, haben diese jeweils $\frac{1}{n}$ der Breite des Knotens.

Der Teilbaum ist der Abschnitt des Baumes mit dem Knoten als Wurzel in Klammernotation

2 Umsetzung

2.1 Klasse Knoten

Zuerst habe ich eine Klasse für alle Knoten implementiert, diese haben also die Attribute *width* (Breite), *depth* (Tiefe), die bei 0 für die Wurzel startet und größer wird, und *subtree_str* (Teilbaum), der den Teilbaum des Knotens als String in Klammernotation beinhaltet

Ich habe den gespeicherten Wert die Breite invertiert, da die relative Breite immer ein Bruch mit dem Zähler 1 ist. Die Breite der Wurzel eines Baumes ist immer 1 und diese wird immer weiter durch ganze Zahlen geteilt, da es nur eine natürliche Anzahl an Kindknoten geben kann. Die gespeicherte Breite, also $Knoten.Breite = rel.Knotenbreite^{-1}$ ist dann immer eine natürliche Zahl und deshalb ein Integer.

Die Methode __eq__() ermöglicht den Vergleich von 2 Knoten mit dem == Operatoren. Dies ist wichtig, wenn die Knoten einzeln bei der Prüfung der Drehfreudigkeit mit ihrem gegenüberstehendem Knoten in Höhe und Breite verglichen werden.

2.2 Zerlegung der Teilbaum-String in ihre Teilbäume

Die Funktion find_subtrees() zerlegt den Teilbaumstring vom Parameter *subtree_str* in all seine Kind-Teilbäume. Ein Teilbaum kommt jeweils zustande, wenn alle geöffneten Klammern wieder von einer geschlossenen Klammer geschlossen werden, da es dann ein ganzer Kindknoten mit Teilbaum ist. Der ganze Teilbaumstring wird also Klammer für Klammer durchgangen, der Zähler *bracket_counter* wird für jede geöffnete Klammer erhöht und für jede geschlossene erniedrigt. Wenn der Wert des Zählers 0 erreicht, gibt es gleich viele geöffnete, wie geschlossene Klammern, es ist ein fertiger Teilbaum eines Kindknotens.

Der Index des Anfangs wird immer in *child_node_start* gespeichert, das Ende ist der Index der Klammer, bei der *bracket_counter* 0 wird. Der Abschnitt dazwischen ist dann der Kindknoten und wird mit dem Slicing von *subtree_str* mit [*child_node_start* + 1 : *bracket_index*] ausgewählt.

Dieser Kindknoten-Teilbaum wird dann in der Liste *child_nodes_subtrees* hinzugefügt.

Diese List beinhaltet nach einem Durchlauf dann alle Teilbäume ohne die äußeren Klammern in geordneter Reihenfolge, da man durch *subtree_str* auch von Links nach Rechts geht.

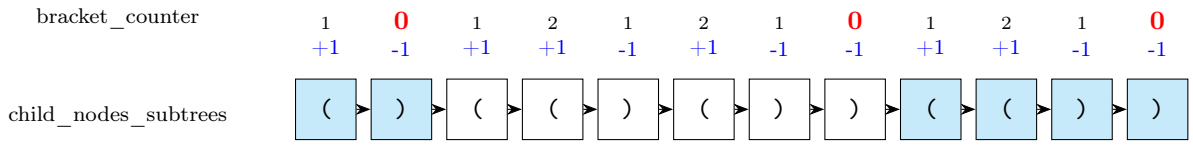


Abb. 2: Der Teilbaum "()(()())(())"wird von find_subtrees() durchgangen, Wenn bracket_counter 0 wird fängt ein neuer Teilbaum an (hier farbig markiert). Die Liste der zurückgegebenen Teilbäume child_nodes_subtrees ist `["", "()()", "()"]`*, da die äußeren Klammern des Teilbaumes beim Slicing entfernt werden*

2.3 Rekursive Durchsuchung des Baumes

Die rekursive Durchsuchung des Baumes erfolgt durch die Funktion *find_leaves()* bzw. innerhalb davon mit *search()*. Die Funktion *find_leaves()* erhält bei der Ausführung die Wurzel des gesamten Baumes als Parameter, wovon die Liste aller Blätter ermittelt werden soll. Diese Liste wird in *leaves* in *find_leaves()* gespeichert, da diese Funktion nur einmal aufgerufen wird und deshalb bei dem Rekursiven Aufruf nicht wieder initialisiert wird.

Die wahre rekursive Funktion ist also *search()*, die einmal mit der Wurzel als Argument in *find_leaves()* aufgerufen wird. Darin werden die Teilbäume der Kindknoten des Argumentes mit *find_leaves()* des Teilbaumes ermittelt. Dies erfolgt durch *find_subtrees()*. Die Kindknoten werden anschließend daraus konstruiert, dass die Tiefe des Kindknotens um eins größer ist als die des Elternknotens, die Breite des Kindknotens die Breite des Elternknotens mit der Anzahl der Kindknoten multipliziert ist. (Die Breite wird immer invertiert gespeichert, darum multipliziert) Jede Teilbaum-String hat dann ihren eigenen Knoten.

Knoten, die einen leeren Teilbaum haben, und deshalb keine Kindknoten haben, sind Blätter und werden dann direkt der Liste aller Blätter *leaves* aus *find_leaves()* hinzugefügt.

Jeder anderen Kindknoten wird direkt durch einen rekursiven Aufruf der Methode *search()* weiter auf seine Kindknoten untersucht, die weiter erforscht werden. Die Funktion *search()* endet erst dann, wenn alle von allen Kindknoten erforscht wurden. Alle Blätter sind dann in *leaves* sind.

2.4 Ausführung des Programms

Bei der Ausführung des Programms ist die Bedingung *if __name__ == "__main__":* wahr. Der Code darunter wird anschließend ausgeführt. Darin wird die Datei ausgelesen, die als Terminalargument eingegeben wurde (*sys.argv[1]*). Es wird dann die Wurzel direkt bei *find_leaves* als Parameter eingeben, die daraus resultierende geordnete Liste aller Blätter wird dann in *leaves1* gespeichert wird. Die Symmetrie wird anschließend durch den Vergleich mit einer umgekehrten Kopie verglichen. Wenn die beiden Listen übereinstimmen, ist der Baum drehfreudig. Dies wird auf dem Terminal ausgegeben.

Wenn der Baum drehfreudig ist, wird auch eine .svg-Grafik erstellt um dies zu veranschaulichen. Der Pfad davon wird auch auf dem Terminal gedruckt, die Datei erscheint auch im Ordner Aufgabe1_Bilder mit dem selben Namen, wie die Textdatei, in der der Baum war.

Die Ausführung aus dem Computer-Terminal erfolgt so, dass man sich erst im Verzeichnis des Programms Aufgabe1.py befinden muss. Das Programm wird durch den Befehl *python Aufgabe1.py eingabedatei.txt* gestartet. *eingabedatei.txt* wird dabei mit dem Pfad der Datei ersetzt, in der der Baum niedergeschrieben ist, bei dem die Drehfreudigkeit geprüft werden soll. Man soll aber darauf achten, dass der Python-Interpreter sich im PATH befindet.

3 Werkzeuge

Es wurde keine KI bei der Ideengewinnung oder bei der Umsetzung benutzt, ich habe jedoch die kostenlose ChatGPT-Version ausschließlich zur Überprüfung des Codes auf Leichtsinnsfehler benutzt. Bei Unklarheiten, wie etwas in Python implementiert wird, wie z.B. Terminalargumente oder die einfache Umkehrung einer Liste habe ich Hilfe bei StackOverflow gefunden.

Um die Terminalargumente zu implementieren habe ich die Library sys in python importiert. *sys.argv[1]* nimmt dann die nächste Terminaleingabe nach der Ausführungsdatei, die dann in meinem Code geöffnet wird.

Für die Visualisation des drehfreudigen Baumes habe ich die Python-Library *svgwrite* benutzt, denn sie erstellt eine Datei, die mit allen Browsern kompatibel und vektorbasiert ist, also ohne Qualitätsverlust recht klein ist. Damit kann man also mit einfachen Python-Code schreiben, bei dem in eine .svg-Datei Formen eingefügt werden. Die Visualisierung habe ich in der Dokumentation nicht eingeschlossen, da sie nicht zur Lösung des zentralen Problems gehört [1].

4 Beispiele

Beispiele von der BWINF-Seite. Rot markierte Beispiele sind nicht drehfreudig, Grün markierte sind drehfreudig. Bei den ausgegebenen Dateipfaden wurde der Pfadpräfix entfernt, da er von dem System und der Umgebung abhängig ist.

4.1 Konsolenausgaben

drehfreudig01.txt

True
Der Baum ist Drehfreudig
`<...>\Aufgabe1_Bilder\drehfreudig01.svg`

drehfreudig02.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig03.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig04.txt

True
Der Baum ist Drehfreudig
`<...>\Aufgabe1_Bilder\drehfreudig04.svg`

drehfreudig05.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig06.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig07.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig08.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig09.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig10.txt

True
Der Baum ist Drehfreudig
`<...>\Aufgabe1_Bilder\drehfreudig10.svg`

drehfreudig11.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig12.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig13.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig14.txt

False
Der Baum ist nicht Drehfreudig

drehfreudig15.txt

False
Der Baum ist nicht Drehfreudig

[1] https://bwinf.de/bundeswettbewerb/haeufige-fragen/ [13.11.2025]

4.2 Bilder

Hier kommen die ausgegebene Diagramme als Bild, sie sind aber im Ordner Aufgabe1_Baum enthalten.

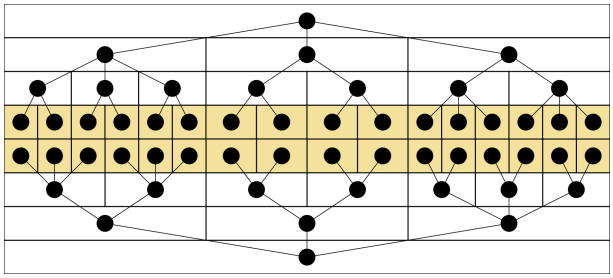


Abbildung 1: drehfreudig01.svg

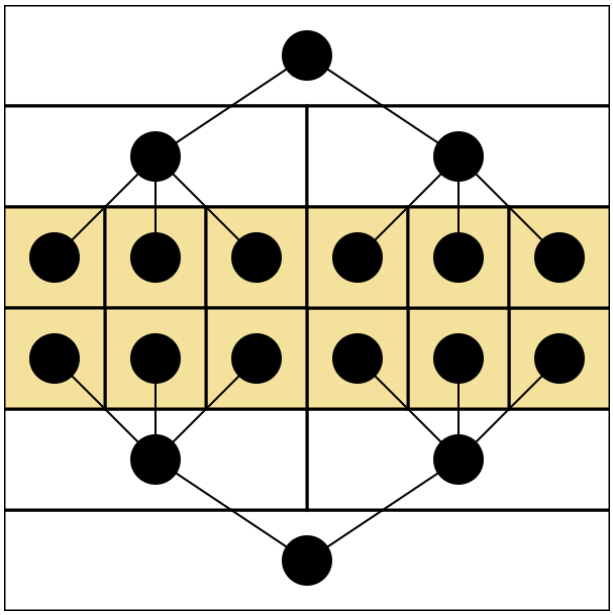


Abbildung 2: drehfreudig04.svg

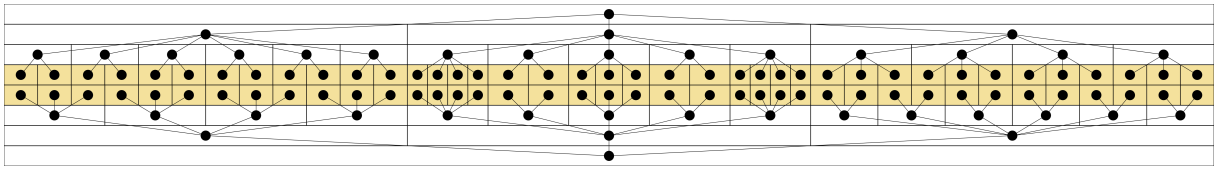


Abbildung 3: drehfreudig10.svg

5 Quellcode

5.1 Klasse für Knoten

```
class Node:
    """Objekt fuer Knoten im Baum mit Breite 1/relative Breite, Teilbaum und Tiefe"""

    def __init__(self, width, depth, subtree_str):
        self.width = width
        self.depth = depth
        self.subtree_str = subtree_str
```

```
    # Ermoeglicht Vergleich von Knoten, bei Prueufung der Symmetrie nuetzlich
    def __eq__(self, other):
        if not isinstance(other, Node):
            return NotImplemented
        return (self.width == other.width and
                self.depth == other.depth and
                self.subtree_str == other.subtree_str)
```

5.2 Zerlegung der Teilbaum-String in ihre Teilbäume

```
def find_subtrees(subtree_str):
    """Zerlegt einen Teilbaumstring in Kind-Teilbaeume"""
    child_nodes_subtrees = []
    bracket_counter = 0
    child_node_start = 0

    for bracket_index in range(len(subtree_str)):
        if subtree_str[bracket_index] == "(":
            if bracket_counter == 0:
                child_node_start = bracket_index
            bracket_counter += 1
        elif subtree_str[bracket_index] == ")":
            bracket_counter -= 1
            if bracket_counter == 0:
                child_nodes_subtrees.append(subtree_str[child_node_start + 1 : bracket_index])
    return child_nodes_subtrees
```

5.3 Rekursive Durchsuchung des Baumes

```
def find_leaves(root_node):
    """Gibt Liste aller Blaetter in Reihenfolge von links nach rechts zurueck"""
    leaves = []

    def search(node):
        """Durchsucht rekursiv den Teilbaum, f\"ugt Kindknoten zu leaves hinzu,
        wenn Kindknoten ein Blatt ist"""
        child_nodes_subtrees = find_subtrees(node.subtree_str)

        for child_subtree in child_nodes_subtrees:
            if child_subtree == "":
                leaves.append(Node(node.width * len(child_nodes_subtrees), node.depth + 1, ""))
            else:
                search(Node(node.width * len(child_nodes_subtrees), node.depth + 1, child_subtree))

    search(root_node)
    return leaves
```

5.4 Prüfung der Drehfreudigkeit

Ausgabe ist

```
leaves1 = find_leaves(Node(1, 0, tree[1:-1]))

    # prueft, ob umgekehrter Baum derselbe ist
    if leaves1 == leaves1[::-1]:
        # Baum drehfreudig
    else:
        # Baum nicht drehfreudig
```